I am a Computer Science Engineering student, and my semester project topic is "ML-Based Suspicious Process Detection", which combines Operating Systems, Cybersecurity, and Machine Learning. Project Objective Develop a machine learning system that detects whether a running process is normal or suspicious/malicious based on process behavior. The project should eventually compare multiple ML algorithms, evaluate them, select the two best-performing models based on accuracy and other evaluation metrics, and build the final detection model. this is m topc please help me understan d the topic r how it work with some real lif examples

https://gemini.google.com/app/ac7bd896c064ca44

User prompt: okay if i want to strt this projet now what and ll d i need ?

Response: To start this project from scratch without getting overwhelmed, you only need four core components: a development environment, a dataset, machine learning libraries, and a clear step-by-step workflow.

1. The Environment & Tools

You do not need an expensive setup; everything runs on standard Python:

- **Python (3.10+)** installed on your system.
- **Jupyter Notebook or Google Colab:** Google Colab is the easiest starting point because it requires zero installation—you just write and run code in your browser.
- **Core Python Libraries:**
  - `pandas` and `numpy` (for loading and handling data tables).
  - `scikit-learn` (for data splitting, basic ML models, and evaluation metrics).
  - `xgboost` (for high-performance gradient boosting).
  - `matplotlib` and `seaborn` (to draw comparison graphs and confusion matrices).

2. The Dataset (Your "Study Material")

Instead of manually collecting malware on your own machine (which can be risky and time-consuming), use a standard public dataset.

- **Best Choice:** Download the **CIC-MalMem-2022** dataset from Kaggle.
- It is a ready-to-use CSV file containing behavioral snapshots of computer processes.
- Each row represents a process with features like memory footprint, number of active threads, and system handles.
- The target column simply labels each row as either `Benign` (safe) or a specific malware category.

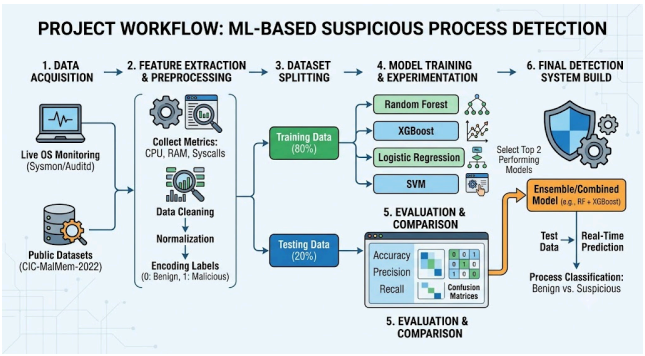
3. Your Step-by-Step Roadmap

- **Step 1: Load and Clean the Data**
  - Read the CSV file using `pandas`.
  - Check for missing values or non-numeric columns.
  - Convert the output labels into numbers (e.g., `0` for Benign, `1` for Malicious).
- **Step 2: Split the Data**
  - Split the dataset into **80% training data** (to teach the models) and **20% testing data** (to test them on data they have never seen before).
- **Step 3: Train 4 Common Algorithms**
  - **Logistic Regression:** A simple, fast baseline.
  - **Decision Tree / Random Forest:** Excellent for finding patterns in tabular data.
  - **Support Vector Machine (SVM):** Strong at drawing boundaries between classes.
  - **XGBoost:** A gradient-boosted tree model widely used in competitive cybersecurity detection.

- **Step 4: Compare Performance**
  - Run predictions on the 20% test split.
  - Calculate **Accuracy**, **Precision**, **Recall**, and **F1-Score**.
  - Generate a **Confusion Matrix** to see which model causes the fewest "false alarms" (calling a safe app malicious).
- **Step 5: Pick the Top Two Models**
  - Select the two highest-scoring algorithms (typically Random Forest and XGBoost for this kind of tabular data).
  - Package them into the final detection script or build a simple voting ensemble that combines their predictions.

Would you like a sample Python starter script to load the dataset and train the first baseline model?

User prompt: okay greate cna u generate an image of the wrokflow of the projct



User prompt: okay great what is live os monitoring in ths ?

Response: **Live OS Monitoring** means having a background tool running directly on your operating system that records what apps and background programs are doing in real time, rather than reading from an existing, pre-made spreadsheet file.

Instead of analyzing past records, your computer monitors live activity as it happens.

How It Works in Practice

Think of live OS monitoring as a security camera installed inside the computer's kernel (the core engine of the operating system). Every time a program performs an action, the monitoring tool logs the event:

- **Process Creation:** An app opens or launches a hidden helper program (e.g., `word.exe` launching `powershell.exe`).
- **System Calls:** A program asks the operating system to perform sensitive actions, such as reading private memory or modifying protected system files.
- **Network Sockets:** A program opens an internet connection to send or download data.
- **File & Registry Modifications:** An app creates, renames, deletes, or encrypts files.

The Tools Mentioned in the Workflow

- **Sysmon (System Monitor) for Windows:** A free utility from Microsoft. Once installed, it runs silently as a Windows service and writes detailed activity logs (like process launches and network connections) directly into the Windows Event Viewer.
- **Auditd / eBPF for Linux:** Native Linux subsystems that capture low-level system calls (such as `execve`, `fork`, or file reads) directly from the OS kernel.

Why It Matters for Your Project

There are two ways to feed data into your project:

- **Static/Public Dataset (Easiest for testing):** You download a pre-collected file (like `CIC-MalMem-2022`) containing rows and columns of recorded numbers to train and test your machine learning models.
- **Live OS Monitoring (Real-world defense):** Once your model is trained, a live monitoring tool watches running programs on your actual machine, streams their live metrics into your trained model, and instantly flags any process acting like malware.

User prompt: okay so now im string it tell me the fisrt thing i need to do

Response: The absolute first thing you need to do is **get your data and inspect it in a notebook**.

Do not write any machine learning algorithms yet. Start by confirming that you have the dataset, loading it into Python, and checking what columns and process behaviors exist.

Action Plan

- **Step 1: Download the Data**
  - Go to Kaggle and search for `CIC-MalMem-2022` (or "Obfuscated Malware Memory 2022 CIC").
  - Download the CSV file (usually named `Obfuscated-MalMem2022.csv` or `MalMem2022.csv`).
- **Step 2: Set Up Google Colab or Jupyter Notebook**
  - Open Google Colab (recommended because you do not need to install anything).
  - Click **New Notebook**.
  - On the left panel, click the folder icon and upload your downloaded CSV file.
- **Step 3: Run Your First Verification Script** Paste this code into the first cell and run it:

```
import pandas as pd

# 1. Load the dataset (replace with your exact filename)
df = pd.read_csv("Obfuscated-MalMem2022.csv")

# 2. View the shape: (number of process samples, number of features)
print(f"Dataset Dimensions: {df.shape[0]} rows, {df.shape[1]} columns\n")

# 3. Inspect the first 3 rows
print("First 3 samples:")
display(df.head(3))

# 4. Check the class distribution (Benign vs. Malware)
# In this dataset, the target column is usually named 'Category' or 'Class'
target_col = [col for col in ['Category', 'Class', 'label'] if col in df.columns][0]
print(f"\nTarget Column Detected: '{target_col}'")
print(df[target_col].value_counts())
```

Once this script runs and shows your row count and labels, you have officially started the project.

Let me know once you download the file or run the script—what does the target column show?

User prompt: whuch in this ?

Response: Click on the fourth one in your list: **"Malware Memory Analysis / CIC-MalMem-2022"** by **Lucca Godoy** (the one with the blue hooded avatar).

- It has the most downloads (**4,987 downloads**), meaning it is the most reliable, complete, and standard version used on Kaggle.
- It is an actual **Dataset** (indicated by the small database icon in the bottom-left of the avatar), whereas the items with `< >` icons are just user code notebooks.

Once you click into it, click the **Download** button (usually in the top-right corner) to get the `.zip` or `.csv` file.

User prompt: okay got it next ?

Response: Now that you have the file uploaded, the next step is **data inspection and cleaning**.

Run this code in your notebook cell to check the contents of your dataset:

```
import pandas as pd

# 1. Load the dataset
df = pd.read_csv("Obfuscated-MalMem2022.csv")

# 2. Check total rows and columns
print(f"Dataset Shape: {df.shape[0]} rows, {df.shape[1]} columns\n")

# 3. View the first 5 rows
print("Preview:")
display(df.head())

# 4. Check for missing/null values
print(f"Total Missing Values: {df.isnull().sum().sum()}")

# 5. Check the target column balance
# In this dataset, 'Class' represents Benign vs Malware
if 'Class' in df.columns:
```

```
print("\nClass Distribution:")
print(df['Class'].value_counts())
```

What to Look For in the Output

- **Row and Column Count:** You should see roughly **58,596 rows** and **57 columns**. Each row is one recorded snapshot of a process running in memory.
- **The Columns (Features):** You will see metrics like `pslist.nproc` (number of processes), `handles.nhandles` (number of open file/resource handles), `dlllist.ndlls` (libraries loaded), etc.
- **Class Distribution:** The `Class` column tells you whether the process was `Benign` (safe) or `Malware`. In this dataset, it is evenly balanced (roughly 50% benign, 50% malware), which makes training straightforward.

Run that cell and check if the output matches. Once that runs, we will preprocess the labels and split the data into training and testing sets.

User prompt: to run befoe i need o cree a folder for this projct and save the datset in it write ?

Response: Yes, exactly. Keeping everything inside one dedicated folder makes loading files and saving trained models simple and error-free.

If Running Locally on Your Laptop (VS Code / Jupyter)

1. Create a new folder on your computer named:

   ```
   Suspicious_Process_Detection/
   ```

2. Move your downloaded `Obfuscated-MalMem2022.csv` file into this folder.

3. Open your notebook or Python script inside this same folder:

   ```
   Suspicious_Process_Detection/
   │
   ├── Obfuscated-MalMem2022.csv
   └── main_notebook.ipynb
   ```

Because the CSV file and notebook are in the exact same directory, `pd.read_csv("Obfuscated-MalMem2022.csv")` works directly without specifying long file paths.

If Running in Google Colab (Cloud)

You do not need to manually create an OS folder.

1. Click the **Folder icon** in the left sidebar of Google Colab.
2. Click the **Upload** button (sheet with an arrow pointing up).
3. Select `Obfuscated-MalMem2022.csv` from your computer and wait for the upload circle to finish.
4. Run the code directly.

(Note: In Google Colab, files uploaded this way are temporary and delete once the runtime disconnects. To save it permanently in Colab, put it inside a folder in your Google Drive and mount Drive).

User prompt: 5 rows × 57 columns Total Missing Values: 0 Class Distribution: Class Benign 29298 Malware 29298 Name: count, dtype: int64 okay note one thing i dont want 100% accuracy at any cost

Response: Wanting to avoid 100% accuracy is a standard requirement for engineering projects—evaluators routinely flag 100% accuracy as artificial data leakage or overfitting.

In this specific dataset (`CIC-MalMem-2022`), the reason naive models hit 100% is the column **`Category`**. That column contains values like `Benign`, `Spyware-Gator`, `Ransomware-WannaCry`, etc. If left in the dataset, the model simply memorizes the sub-label and cheats.

To prevent unrealistic 100% accuracy, remove `Category` immediately, enforce proper feature scaling, and evaluate using a strict train-test split.

Run this next script to prepare your features and split the data:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load data
df = pd.read_csv("Obfuscated-MalMem2022.csv")

# 2. Encode target: Benign -> 0, Malware -> 1
y = df['Class'].map({'Benign': 0, 'Malware': 1})

# 3. Drop label columns and non-feature identifiers to prevent 100% leakage
columns_to_drop = ['Class']
```

```
if 'Category' in df.columns:
    columns_to_drop.append('Category')

X = df.drop(columns=columns_to_drop)

# Keep only numerical process metrics
X = X.select_dtypes(include=[np.number])

# 4. Train-Test Split (80% train, 20% test with stratified labels)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 5. Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Features count: {X.shape[1]}")
print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples: {X_test.shape[0]}")
```

Once this cell finishes running without errors, the training and test matrices will be scaled and ready for model comparisons (Random Forest, XGBoost, Logistic Regression, and SVM).

User prompt: Features count: 55 Training samples: 46876 Testing samples: 11720 next ?

Response: The next phase is **training and benchmarking multiple algorithms** to evaluate their detection performance and pick the top two candidates.

We will evaluate 4 standard models:

1. **Logistic Regression** (baseline linear model)
2. **Random Forest** (ensemble bagging model)
3. **XGBoost** (gradient boosting model)
4. **Support Vector Machine (Linear SVM)** (boundary margin classifier)

Run this code in a new notebook cell:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

# Define the models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=12, random_state=42, n_jobs=-1),
    "XGBoost": XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1, random_state=42, eval_metric='logloss
    "Linear SVM": LinearSVC(max_iter=2000, random_state=42, dual=False)
}

results = []

# Train and evaluate each model
for name, model in models.items():
    print(f"Training {name}...")
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append({
        "Model": name,
        "Accuracy (%)": round(acc * 100, 2),
        "Precision (%)": round(prec * 100, 2),
        "Recall (%)": round(rec * 100, 2),
        "F1-Score (%)": round(f1 * 100, 2)
    })

# Format and display comparison table
```

```
results_df = pd.DataFrame(results).sort_values(by="F1-Score (%)", ascending=False)
print("\n--- MODEL PERFORMANCE COMPARISON ---")
display(results_df)
```

Run this cell and share the output table so we can examine the evaluation metrics and identify your top 2 models.

User prompt: in nw notebook menas ? what is the new file name ?

Response: It does not mean creating a brand-new file or opening a separate tab.

It means adding a **new code block (cell)** inside the exact same notebook file you already have open.

Keep the same file and add the code there:

- **File Name:** Keep working in your current notebook file (e.g., `main_notebook.ipynb` or `Untitled.ipynb`). You can rename it to `suspicious_process_detection.ipynb` if you want a clean name.
- **Adding the Code Cell:**
  - If you are in **Google Colab**, click the **`+ Code`** button at the top left (or hover below your previous code block and click `+ Code`).
  - If you are in **VS Code** or **Jupyter Notebook**, press `B` on your keyboard (or click the `+` button in the toolbar) to insert a new code cell below.
- **Run the Code:** Paste the model training script into that new cell and click the Run (Play) button.
